MEMORIA TÉCNICA — HOGWARTS

Catálogo interactivo de personajes del universo de Harry Potter

Proyecto individual de JavaScript de Carmen Asensio

Tecnologías: HTML5, CSS3, JavaScript (módulos ES), JSON, Harry Potter API y localStorage.

1. Descripción del proyecto

Hogwarts es una aplicación web multipágina que permite explorar el universo de Harry Potter a través de un catálogo interactivo de personajes. La aplicación permite consultar información de los personajes, conocer sus características principales y descubrir a qué casa de Hogwarts pertenecen.

Además, incorpora funciones de búsqueda, filtrado por casa y paginación para facilitar la exploración del catálogo. También permite guardar personajes favoritos y consultarlos posteriormente en una sección específica.

La información se obtiene mediante una API pública y se presenta en una interfaz diseñada para ofrecer una experiencia visual relacionada con el universo de Harry Potter.

1.1 Objetivos de aprendizaje

- Desarrollar una aplicación web multipágina utilizando HTML5, CSS3 y JavaScript vanilla.
- Aplicar JavaScript moderno mediante módulos ES (`import` y `export`).
- Aplicar conceptos de programación orientada a objetos mediante la creación de clases.
- Consumir una API REST pública y trabajar con datos en formato JSON.
- Gestionar información de forma asíncrona.
- Manipular elementos HTML dinámicamente mediante el DOM.
- Implementar funcionalidades de búsqueda, filtrado y paginación.
- Utilizar `localStorage` para guardar información en el navegador y conservar los favoritos entre sesiones.
- Diseñar una interfaz adaptable a diferentes tamaños de pantalla.

1.2 Alcance

Dentro del alcance	Fuera del alcance
Consulta dinámica de personajes mediante una API pública.	Registro y autenticación de usuarios.
Visualización de personajes mediante tarjetas.	Base de datos propia para almacenar información.
Búsqueda de personajes por nombre.	Gestión de perfiles de usuario.
Filtrado de personajes por casa de Hogwarts.	Favoritos sincronizados entre distintos dispositivos.
Navegación por el catálogo mediante paginación.	Modificación de la información original de los personajes.
Guardado y consulta de personajes favoritos mediante `localStorage`.	

2. Requisitos de la aplicación

2.1 Requisitos funcionales

ID	Requisito	Ubicación orientativa
RF-01	Consultar los personajes obtenidos	`utils/api.js`

	desde la API pública de Harry Potter.	
RF-02	Mostrar la información de los personajes mediante tarjetas.	`ui/renderer.js`
RF-03	Buscar personajes por su nombre.	`core/catalog.js`
RF-04	Filtrar los personajes según su casa de Hogwarts.	`core/catalog.js`
RF-05	Navegar por el catálogo mediante paginación.	`core/catalog.js`
RF-06	Marcar y desmarcar personajes como favoritos.	`ui/renderer.js` · `utils/localStorage.js`
RF-07	Guardar los favoritos en el almacenamiento local del navegador.	`utils/localStorage.js`
RF-08	Consultar los personajes guardados como favoritos.	`core/favorite.js`
RF-09	Eliminar personajes de la lista de favoritos.	`core/favorite.js`
RF-10	Navegar entre las diferentes páginas de la aplicación.	Archivos HTML

2.2 Requisitos no funcionales

- **RNF-01. Organización modular:** utilización de módulos ES de JavaScript para separar las funcionalidades.
- **RNF-02. Separación de responsabilidades:** distribución del código en carpetas específicas para la lógica, la interfaz, los modelos y las utilidades.
- **RNF-03. Diseño adaptable:** interfaz responsive que permita utilizar la aplicación en diferentes dispositivos.
- **RNF-04. Persistencia local:** almacenamiento de los personajes favoritos mediante `localStorage`.
- **RNF-05. Mantenibilidad:** estructura organizada para facilitar la comprensión, modificación y ampliación del código.
- **RNF-06. Interfaz intuitiva:** presentación clara de la información y controles de navegación y búsqueda.

3. Tecnologías y herramientas

Para el desarrollo de Hogwarts se han utilizado las siguientes tecnologías y herramientas:

Tecnología / herramienta	Utilización
HTML5	Estructura y contenido de las diferentes páginas de la aplicación.
CSS3	Diseño visual, distribución de elementos, estilos y adaptación responsive.
JavaScript	Lógica de la aplicación, interacción con el usuario y manipulación del DOM.
Módulos ES	Organización del código mediante `import` y `export`.
JSON	Formato de intercambio y tratamiento de los datos obtenidos desde la API.
Harry Potter API	Fuente externa de información de los personajes del universo de Harry Potter.
localStorage	Almacenamiento persistente de los personajes favoritos en el

	navegador.
Font Awesome	Incorporación de iconos en la interfaz.
Git / GitHub	Control de versiones y gestión del código fuente.
Visual Studio Code	Entorno de desarrollo.

3.1 API utilizada

La aplicación utiliza la API pública **Harry Potter API**, disponible en:

https://hp-api.onrender.com/api/characters

Esta API proporciona información de los personajes del universo de Harry Potter, incluyendo datos personales, características mágicas, pertenencia a una casa de Hogwarts e imágenes.

Entre los campos utilizados por la aplicación se encuentran:

- Nombre y nombres alternativos.
- Especie y género.
- Casa de Hogwarts.
- Fecha y año de nacimiento.
- Ascendencia.
- Color de ojos y cabello.
- Información de la varita.
- Patronus.
- Condición de estudiante o miembro del personal de Hogwarts.
- Actor o actriz que interpreta al personaje.
- Imagen del personaje.

La información recibida se procesa en JavaScript y se utiliza para construir las tarjetas que se muestran en el catálogo.

4. Estructura y arquitectura del proyecto

La aplicación se organiza en diferentes carpetas para separar la estructura HTML, los estilos, la lógica de funcionamiento, los modelos de datos y las funciones auxiliares.

4.1 Estructura de carpetas

```
Hogwarts/
│
├── index.html
│
├── html/
│   ├── catalog.html
│   └── favorite.html
│
├── css/
│   ├── variable.css
│   ├── style.css
│   ├── catalog.css
│   └── favorite.css
│
├── core/
│   ├── index.js
│   ├── catalog.js
│   └── favorite.js
```

```
│
├── model/
│   └── Character.js
│
├── ui/
│   └── renderer.js
│
├── utils/
│   ├── api.js
│   ├── error.js
│   └── localStorage.js
│
└── img/
    └── wizard.jpg
```

4.2 Organización de módulos y responsabilidades

La estructura del proyecto permite distribuir las funcionalidades en módulos independientes, evitando concentrar toda la lógica de la aplicación en un único archivo.

Carpeta / archivo	Responsabilidad
`index.html`	Página principal de presentación de Hogwarts y sus cuatro casas.
`html/catalog.html`	Página que contiene el catálogo de personajes, los filtros y la paginación.
`html/favorite.html`	Página destinada a mostrar los personajes guardados como favoritos.
`css/`	Hojas de estilo de las diferentes páginas y variables visuales comunes.
`core/index.js`	Gestión del menú de navegación de la página principal.
`core/catalog.js`	Carga del catálogo, búsqueda, filtrado, paginación y presentación de personajes.
`core/favorite.js`	Recuperación y visualización de los personajes favoritos.
`model/Character.js`	Definición de la clase que representa a los personajes.
`ui/renderer.js`	Creación dinámica de formularios y tarjetas mediante el DOM.
`utils/api.js`	Comunicación con la API y obtención de los datos.
`utils/error.js`	Módulo destinado a la gestión de errores.
`utils/localStorage.js`	Funciones para consultar, guardar, eliminar y comprobar favoritos.
`img/`	Recursos gráficos utilizados por la aplicación.

4.3 Modelo de clases y programación orientada a objetos

La aplicación utiliza programación orientada a objetos mediante la clase `Character`, definida en `model/Character.js`.

Esta clase representa a cada personaje del universo de Harry Potter y permite transformar los objetos recibidos desde la API en instancias con una estructura común.

4.4 Flujo de datos

La información de los personajes procede de la API pública Harry Potter API.

La aplicación realiza una petición para obtener los datos, los procesa mediante JavaScript y los transforma en instancias de la clase `Character`.

A continuación, los personajes se filtran según los criterios seleccionados por el usuario y se muestran mediante tarjetas generadas dinámicamente.

Para la página de favoritos, el flujo comienza recuperando los personajes almacenados en `localStorage`, que posteriormente se muestran utilizando el mismo sistema de creación de tarjetas.

La aplicación funciona en el lado del cliente (frontend), sin disponer de un backend ni de una base de datos propia.

5. Diseño de interfaz y experiencia de usuario (UI/UX)

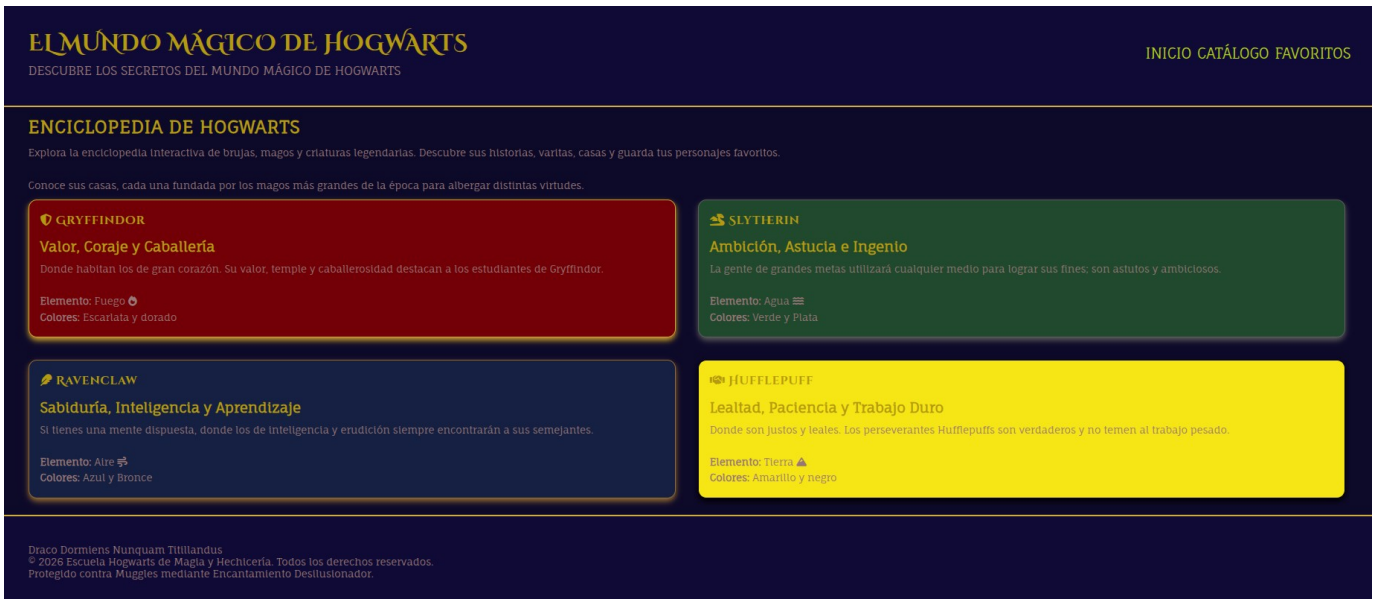
La propuesta visual de Hogwarts busca transmitir la identidad del universo de Harry Potter y de la Escuela de Magia y Hechicería.

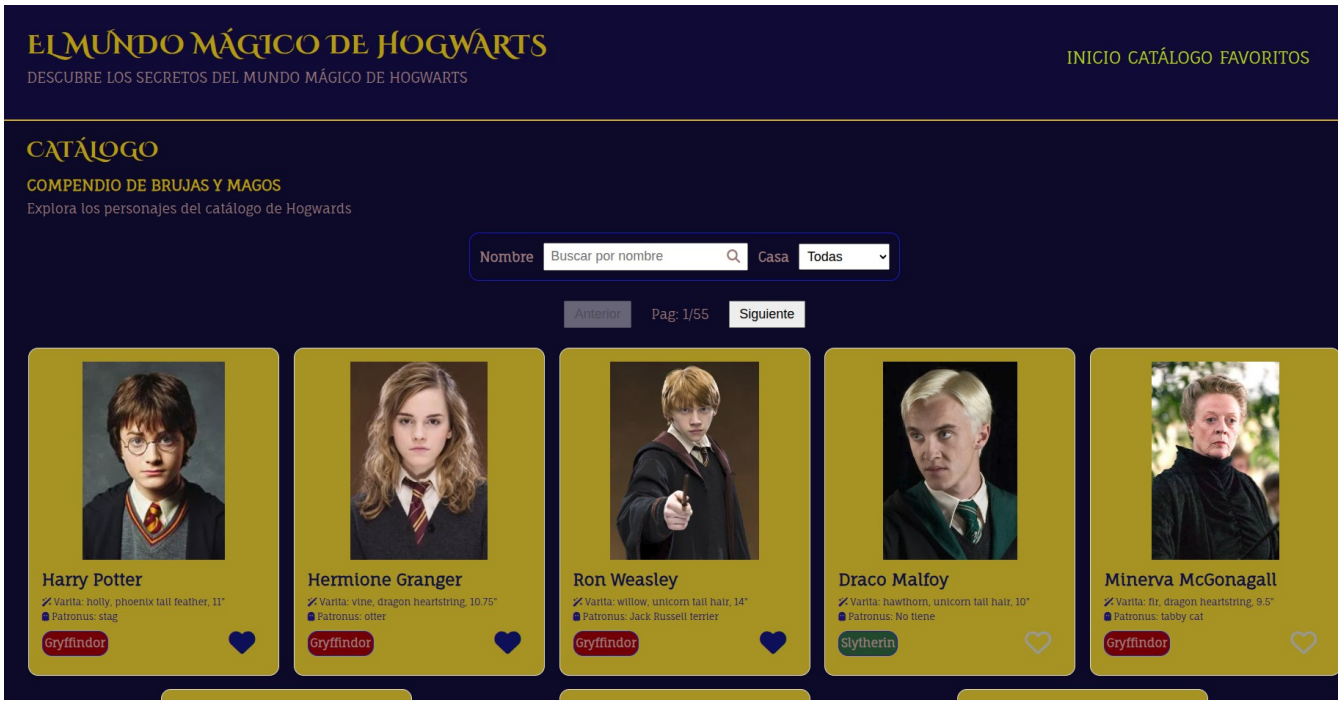
El diseño utiliza elementos visuales asociados a Hogwarts, con especial protagonismo de sus cuatro casas: Gryffindor, Slytherin, Ravenclaw y Hufflepuff.

La página principal presenta las casas mediante bloques diferenciados, mostrando sus características, valores, elementos y colores representativos.

El catálogo utiliza tarjetas de personajes para facilitar la consulta de la información. Cada tarjeta presenta los datos principales del personaje y permite gestionar su estado de favorito.

La interfaz incorpora controles de búsqueda por nombre y selección de casa, así como botones de paginación para facilitar la navegación por el catálogo.





5.1 Identidad visual

La identidad visual se basa en la temática de Hogwarts y en los elementos característicos de sus cuatro casas.

Se utilizan variables CSS para centralizar determinados valores de diseño, como colores, tipografías y otros estilos comunes. Esto facilita mantener la coherencia visual y realizar modificaciones sin tener que cambiar los estilos individualmente en cada componente.

5.2 Navegación y adaptación responsive

La aplicación dispone de un menú de navegación que permite acceder a las diferentes páginas.

En dispositivos con pantallas más pequeñas, el menú se transforma en un menú hamburguesa que permite mostrar u ocultar las opciones de navegación.

También se adaptan los filtros, las tarjetas y los elementos de paginación para facilitar su utilización desde dispositivos móviles.

6. Decisiones técnicas y desafíos

6.1 Selección de la API

La elección de Harry Potter API permite trabajar con información relacionada con una temática concreta y reconocible.

La API proporciona datos de personajes y recursos gráficos que permiten construir un catálogo visual e interactivo.

6.2 Organización modular del código

Una de las decisiones técnicas principales ha sido separar las responsabilidades de la aplicación en diferentes módulos.

Esta organización permite distinguir entre la obtención de datos, el modelo de personajes, la lógica del catálogo, la generación de elementos visuales y la persistencia de favoritos.

De esta forma, cada módulo tiene una función específica y el código resulta más sencillo de mantener y ampliar.

6.3 Gestión de los datos y manipulación del DOM

La información recibida desde la API se transforma en objetos `Character` antes de utilizarse en el catálogo.

Por otro lado, las tarjetas y los controles de búsqueda se generan dinámicamente mediante JavaScript y la manipulación del DOM.

Esta solución permite construir la interfaz a partir de los datos disponibles, sin necesidad de definir previamente una tarjeta HTML para cada personaje.

7. Ampliaciones futuras y conclusiones

7.1 Posibles mejoras

Como posibles ampliaciones del proyecto se plantean las siguientes funcionalidades:

- Incorporar una ficha individual con información más completa de cada personaje.
- Ampliar los criterios de búsqueda y filtrado utilizando otros datos disponibles en la API.
- Mejorar la presentación de los estados de carga y los mensajes de error.
- Incorporar un fichero de configuración en formato **JSON** que permita gestionar los textos de la aplicación en diferentes idiomas y facilitar así su adaptación a un entorno multilenguaje.

Estas propuestas se consideran posibles evoluciones del proyecto y no forman parte de las funcionalidades descritas como implementadas.

7.2 Conclusiones

Hogwarts reúne los conocimientos adquiridos durante el desarrollo de una aplicación web multipágina utilizando HTML5, CSS3 y JavaScript.

El proyecto permite aplicar conceptos de programación modular y orientada a objetos, consumir información de una API pública, trabajar con datos en formato JSON y construir interfaces dinámicas mediante la manipulación del DOM.

También permite implementar funcionalidades habituales en aplicaciones web, como la búsqueda, el filtrado, la paginación y la persistencia de información mediante `localStorage`.

La separación de responsabilidades y la organización del código en módulos facilitan el mantenimiento y permiten plantear nuevas funcionalidades como evolución del proyecto.